CSA1741– Artificial Intelligence

LAB OBSERVATION

NAME: VISHAL.V

REG.NO: 192419033

Experiment No: 1 DATE: 16-10-26

TITLE: Write the python program to solve 8-Puzzle problem

Aim: To write a Python program to solve the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm.

Algorithm:

1. Start the program.
2. Initialize the initial state and goal state.
3. Create a queue and insert the initial state.
4. Remove a state from the queue.
5. Check whether the state matches the goal state.
6. If the goal is reached, print the solution path.
7. Otherwise, generate all possible moves of the empty tile (0).
8. Add unvisited states to the queue.
9. Repeat until the goal is reached.
10. Stop the program.

Program:

```
from collections import deque

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)

goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

q = deque([(start, [])])

visited = {start}

while q:

    state, path = q.popleft()

    path = path + [state]
```

```
    if state == goal:
        for s in path:
            print(s[:3], s[3:6], s[6:])
        break
    z = state.index(0)
    for n in [z-3, z+3, z-1, z+1]:
        if 0 <= n < 9 and not (abs(n-z) == 1 and z//3 != n//3):
            new = list(state)
            new[z], new[n] = new[n], new[z]
            new = tuple(new)
            if new not in visited:
                visited.add(new)
                q.append((new, path))
```

INPUT:

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)

OUTPUT (result):

```
========= RESTART: C:/Users/VISHAL
(1, 2, 3) (4, 0, 6) (7, 5, 8)
(1, 2, 3) (4, 5, 6) (7, 0, 8)
(1, 2, 3) (4, 5, 6) (7, 8, 0)
```

Result: Thus, the Python program to solve the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm was executed successfully, and the goal state was reached.

Experiment No: 2 DATE: 16-10-26

TITLE: Write the python program to solve 8-Queen problem

Aim: To write a simple Python program to solve the 8-Queens problem by placing 8 queens on a chessboard so that no two queens attack each other.

Algorithm:

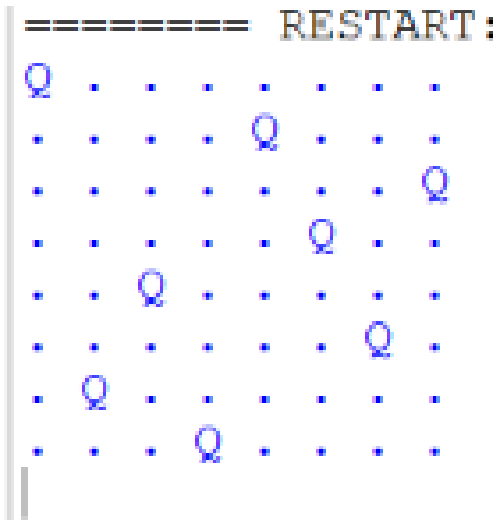
1. Start the program.
2. Place a queen in the first row.
3. Check whether the position is safe.
4. If safe, place a queen in the next row.
5. If no safe position exists, move back and try another position.
6. Repeat until all 8 queens are placed.
7. Print the solution.
8. Stop the program.

Program:

```
def solve(board, row):
    if row == 8:
        return True
    for col in range(8):
        safe = True
        for r in range(row):
            if (board[r] == col or
                abs(board[r] - col) == row - r):
                safe = False
```

```
                break
        if safe:
            board[row] = col
            if solve(board, row + 1):
                return True
    return False
board = [-1] * 8
solve(board, 0)
for row in range(8):
    print(" ".join("Q" if board[row] == col
                else "." for col in range(8)))
```

OUTPUT:



Result: The Python program was executed successfully, placing 8 queens on the chessboard so that no two queens attack each other.

Experiment No: 3 DATE: 16-10-26

TITLE: Write the python program for Water Jug Problem

Aim: To write a Python program to solve the Water Jug Problem using the Breadth-First Search (BFS) algorithm.

Algorithm:

1. Start the program.
2. Initialize two water jugs and the target amount.
3. Create a queue with the initial state (0, 0).
4. Generate possible operations: fill, empty, and pour water between jugs.
5. Add unvisited states to the queue.
6. Check whether either jug contains the target amount.
7. Print the solution when the target is reached.
8. Stop the program.

Program:

```
from collections import deque

a = int(input("Enter capacity of Jug A: "))

b = int(input("Enter capacity of Jug B: "))

target = int(input("Enter target amount: "))

q = deque([(0, 0)])

visited = set()

while q:

    x, y = q.popleft()

    if (x, y) in visited:
```

```
            continue
        visited.add((x, y))
        print(x, y)
        if x == target or y == target:
            print("Target reached")
            break
        q.extend([
            (a, y), (x, b), (0, y), (x, 0),
            (max(0, x-(b-y)), min(b, x+y)),
            (min(a, x+y), max(0, y-(a-x)))
        ])
    else:
        print("No solution")
```

INPUT:

Enter capacity of Jug A: 4

Enter capacity of Jug B: 3

Enter target amount: 2

OUTPUT:

```
======== RESTART: C:/Users/V
Enter capacity of Jug A: 4
Enter capacity of Jug B: 3
Enter target amount: 2
0 0
4 0
0 3
4 3
1 3
3 0
1 0
3 3
0 1
4 2
Target reached
>
```

Result: The Python program was executed successfully, and the Water Jug Problem was solved by finding a sequence of operations to obtain 2 litres of water.

Experiment No: 4 DATE: 21-10-26

TITLE: Write the python program for Cript-Arithmetic problem

Aim: To write a simple Python program to solve a crypt-arithmetic problem in which each letter represents a unique digit.

Algorithm:

1. Start the program.
2. Assign different digits to each letter.
3. Check that the first letter is not zero.
4. Verify whether the equation is correct.
5. Print the solution if the equation is satisfied.
6. Stop the program.

Program:

```
from itertools import permutations

for A, B, C in permutations(range(10), 3):
    if A + B == C:
        print("A =", A, "B =", B, "C =", C)
        break
```

OUTPUT:

```
A = 1 B = 2 C = 3
```

Result: Thus, the crypt-arithmetic problem was solved successfully, with each letter assigned a unique digit.

Experiment No: 5 DATE: 21-10-26

TITLE: Write the python program for Missionaries Cannibal problem

Aim: To write a simple Python program to solve the Missionaries and Cannibals problem using Breadth-First Search (BFS).

Algorithm:

1. Start the program.
2. Initialize 3 missionaries, 3 cannibals, and a boat.
3. Allow the boat to carry 1 or 2 people.
4. Check that cannibals never outnumber missionaries where missionaries are present.
5. Move the boat across the river and record visited states.
6. Repeat until everyone reaches the other side.
7. Print the solution path and stop.

Program:

```
from collections import deque

start = (3, 3, 1)

goal = (0, 0, 0)

moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

q = deque([(start, [])])

visited = set()

while q:

    (m, c, b), path = q.popleft()

    if (m, c, b) == goal:

        for state in path + [(m, c, b)]:
```

```
            print(state)
        break
    if (m, c, b) in visited:
        continue
    visited.add((m, c, b))
    for x, y in moves:
        if b == 1:
            nm, nc, nb = m-x, c-y, 0
        else:
            nm, nc, nb = m+x, c+y, 1
        if 0 <= nm <= 3 and 0 <= nc <= 3:
            if (nm == 0 or nm >= nc) and (nm == 3 or 3-nm >= 3-nc):
                q.append(((nm, nc, nb), path + [(m, c, b)]))
```

INPUT:

- Missionaries: 3
- Cannibals: 3
- Boat capacity: 2
- Initial state: (3, 3, 1)
- Goal state: (0, 0, 0)

OUTPUT:

```
===============
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
```

Result: The Python program finds a sequence of valid boat crossings that transfers all three missionaries and three cannibals safely to the opposite bank.

Experiment No: 6 DATE: 21-10-26

TITLE: Write the python program for Vacuum Cleaner problem

Aim: To write a simple Python program to solve the Vacuum Cleaner problem using a basic AI approach.

Algorithm:

1. Start the program.
2. Enter the status of rooms A and B.
3. Check whether each room is dirty or clean.
4. If a room is dirty, clean it.
5. Move the vacuum cleaner to the next room.
6. Stop when both rooms are clean.

Program:

```
a = input("Enter Room A status (dirty/clean): ").lower()
b = input("Enter Room B status (dirty/clean): ").lower()
pos = input("Enter vacuum position (A/B): ").upper()
while a == "dirty" or b == "dirty":
    if pos == "A":
        if a == "dirty":
            print("Room A cleaned")
            a = "clean"
        else:
            print("Move to Room B")
            pos = "B"
    else:
```

```
    if b == "dirty":
        print("Room B cleaned")
        b = "clean"
    else:
        print("Move to Room A")
        pos = "A"
print("Both rooms are clean")
```

INPUT:

Enter Room A status (dirty/clean): dirty

Enter Room B status (dirty/clean): dirty

Enter vacuum position (A/B): B

OUTPUT:

```
Enter Room A status (dirty/clean): dirty
Enter Room B status (dirty/clean): dirty
Enter vacuum position (A/B): b
Room B cleaned
Move to Room A
Room A cleaned
Both rooms are clean
```

Result: Thus, the Python program successfully cleans both rooms using a simple vacuum cleaner agent.

Experiment No: 7 DATE: 21-10-26

TITLE: Write the python program to implement BFS.

Aim: To write a simple Python program to implement Breadth-First Search (BFS) and traverse a graph level by level.

Algorithm:

1. Start the program.
2. Create a graph with nodes and edges.
3. Enter the starting node.
4. Create a queue and insert the starting node.
5. Remove the first node from the queue.
6. Visit its unvisited neighbouring nodes and add them to the queue.
7. Repeat until the queue becomes empty.
8. Print the traversal order and stop.

Program:

```
from collections import deque
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}
start = input("Enter starting node: ")
```

```
visited = []
queue = deque([start])
while queue:
    node = queue.popleft()
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        for i in graph[node]:
            queue.append(i)
```

Input:

Enter starting node: A

Output:

```
==========================
Enter starting node: A
A B C D E F
```

Result: Thus, the Python program was executed successfully, and the graph was traversed using the Breadth-First Search (BFS) algorithm.

Experiment No: 8 DATE: 21-10-26

TITLE: Write the python program to implement DFS.

Aim: To write a simple Python program to implement Depth-First Search (DFS) and traverse a graph.

Algorithm:

1. Start the program.
2. Create a graph with nodes and edges.
3. Enter the starting node.
4. Mark the starting node as visited.
5. Print the current node.
6. Visit an unvisited neighbouring node recursively.
7. Repeat until all reachable nodes are visited.
8. Stop the program.

Program:

```
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}
visited = set()
def dfs(node):
```

```
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for i in graph[node]:
            dfs(i)
start = input("Enter starting node: ")
dfs(start)
```

Input:

Enter starting node: A

Output:

```
Enter starting node: A
A B D E C F
```

Result: Thus, the Python program was executed successfully, and the graph was traversed using the Depth-First Search (DFS) algorithm.

Experiment No: 9 DATE: 21-10-26

TITLE: Write the python to implement Travelling Salesman Problem.

Aim: To write a simple Python program to solve the Travelling Salesman Problem (TSP) by finding the shortest route to visit all cities and return to the starting city.

Algorithm:

1. Start the program.
2. Enter the number of cities.
3. Enter the distance between each pair of cities.
4. Generate all possible routes.
5. Calculate the total distance of each route.
6. Find the route with the minimum distance.
7. Display the shortest route and its distance.
8. Stop the program.

Program:

```
from itertools import permutations
n = int(input("Enter number of cities: "))
dist = []
for i in range(n):
    row = list(map(int, input().split()))
    dist.append(row)
start = 0
best = 999999
best_path = []
```

```
for path in permutations(range(1, n)):
    route = (start,) + path + (start,)
    cost = 0
    for i in range(n):
        cost += dist[route[i]][route[i+1]]
    if cost < best:
        best = cost
        best_path = route
print("Shortest Route:", best_path)
print("Minimum Distance:", best)
```

Input:

Enter number of cities: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output:

```
Enter number of cities: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Shortest Route: (0, 1, 3, 2, 0)
Minimum Distance: 80
```

Result: Thus, the Python program was executed successfully and the shortest route was found using the Travelling Salesman Problem approach.

Experiment No: 10 DATE: 21-10-26

TITLE: Write the python program to implement A* algorithm.

Aim: To write a simple Python program to implement the A* (A-Star) Search Algorithm to find the shortest path between two nodes in a graph.

Algorithm:

1. Start the program.
2. Create a graph with nodes and distances.
3. Enter the starting node and goal node.
4. Calculate the estimated cost using a heuristic value.
5. Select the node with the lowest total cost.
6. Repeat until the goal node is reached.
7. Display the shortest path and total cost.
8. Stop the program.

Program:

```
import heapq
graph = {
   'A': {'B': 1, 'C': 4},
   'B': {'D': 2, 'E': 5},
   'C': {'F': 3},
   'D': {},
   'E': {'F': 1},
   'F': {}
}
h = {'A': 5, 'B': 4, 'C': 3, 'D': 2, 'E': 1, 'F': 0}
```

```
start = input("Enter start node: ")
goal = input("Enter goal node: ")
q = [(h[start], 0, start, [start])]
while q:
    f, cost, node, path = heapq.heappop(q)
    if node == goal:
        print("Path:", path)
        print("Cost:", cost)
        break
    for next in graph[node]:
        new_cost = cost + graph[node][next]
        heapq.heappush(q, (new_cost + h[next],
                           new_cost, next, path + [next]))
else:
    print("No path found")
```

Input:

Enter start node: A

Enter goal node: F

Output:

```
Enter start node: A
Enter goal node: F
Path: ['A', 'B', 'E', 'F']
Cost: 7
```

Result: Thus, the Python program was executed successfully, and the path from the starting node to the goal node was found using the A* algorithm.